LAPORAN TUGAS BESAR 1
IF3070 DASAR INTELEGENSI ARTIFISIAL
IMPLEMENTASI ALGORITMA LOCAL SEARCH UNTUK MENYELESAIKAN STUDI KASUS



Disusun oleh:
18223103 Muhammad Dhafin Faidhulhaq

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG
2026

DAFTAR ISI

DAFTAR ISI **2**
PENDAHULUAN **3**
1.1 Latar Belakang 3
1.2 Deskripsi Persoalan 3
1.3 Rumusan Masalah 4
1.4 Tujuan 4
LANDASAN TEORI **5**
2.1 Local Search 5
2.2 Hill-Climbing 5
2.3 Simulated Annealing 5
2.4 Genetic Algorithm 5
PEMBAHASAN **5**
3.1 Struktur Program 5
3.2 Implementasi 6
3.2.1 data_uji.json 6
3.2.2 state.py 6
3.2.3 objective.py 7
3.2.4 langkah.py 8
3.2.5 hill_climbing.py 9
3.2.6 simulated_annealing.py 10
3.2.7 genetic_algorithm.py 12
3.2.8 main.py 14
HASIL DAN ANALISIS **15**
4.1 Hasil Eksekusi Program 15
4.2 Hasil Hill Climbing 16
4.3 Hasil Simulated Annealing 16
4.4 Hasil Genetic Algorithm 16
4.5 Hasil Akhir 17
KESIMPULAN DAN SARAN **18**
5.1 Kesimpulan 18
5.2 Saran 18
PUSTAKA **18**
LAMPIRAN **18**

PENDAHULUAN

1.1 Latar Belakang

PT Owen Intim Line merupakan perusahaan yang mengoperasikan kapal kargo pengangkut kendaraan. Perusahaan baru saja memperoleh kontrak pengiriman dari beberapa dealer kendaraan sekaligus. Pada setiap jadwal keberangkatan, perusahaan menerima daftar kendaraan dengan dimensi, berat, tarif pengiriman, dan deadline yang berbeda-beda.

Selama ini, penempatan kendaraan pada geladak kapal dilakukan secara manual. Akibatnya, ruang geladak sering tidak termanfaatkan dengan baik dan keputusan pemilihan letak kendaraan belum tentu menghasilkan keuntungan yang optimal.

Tim operasional meminta dibangun sebuah sistem pendukung keputusan untuk menentukan kendaraan yang akan diangkut serta posisi dan orientasi setiap kendaraan pada geladak. Sistem tersebut harus dapat mengevaluasi berbagai kemungkinan konfigurasi, mempertimbangkan fungsi objektif yang ditentukan, dan menghasilkan solusi penataan yang optimal.

1.2 Deskripsi Persoalan

Kapal dan kendaraan direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Solusi memilih kendaraan yang diangkut dan menentukan posisi serta orientasinya (horizontal atau vertikal) di dalam geladak. Kendaraan yang tidak dipilih berada di luar kapal.

Spesifikasi tugas tidak menyediakan data kapal maupun kendaraan, sehingga kelompok menyusun data uji sendiri. Data ini digunakan secara konsisten pada seluruh algoritma dan seluruh eksperimen supaya hasilnya bisa dibandingkan.

Data kapal

Atribut	Nilai
Dimensi (w x l)	4 x 4
Luas	16 sel
MaxCapacity	7 unit

Data kendaraan

id	Jenis	Dimensi (w x l) (sel)	Weight (unit)	ShippingFee (juta rupiah)	Eta (hari)

motor_1	motor	1 x 2	1	8	2
motor_2	motor	1 x 2	1	6	4
mobilkecil_1	mobil kecil	2 x 2	2	15	3
mobilkecil_2	mobil kecil	2 x 2	2	18	1
mobilbesar_1	mobil besar	2 x 3	3	30	2
mobilbesar_2	mobil besar	2 x 3	3	25	5
Total		24 sel	12 unit	102 juta rupiah	

1.3 Rumusan Masalah

1. Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?
2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?
3. Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?
4. Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?
5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

1.4 Tujuan

1. Mengetahui seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian.
2. Membandingkan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain.
3. Durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya.
4. Menentukan seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan.
5. Menentukan pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm.

LANDASAN TEORI

2.1 Local Search

Local Search adalah kelompok algoritma pencarian yang bekerja dari satu state saat ini dan berpindah ke state tetangganya tanpa menyimpan jalur yang telah dilalui.

2.2 Hill-Climbing

Hill Climbing adalah algoritma *local search* yang secara berulang berpindah ke *neighbor* yang lebih baik daripada *state* saat ini dan berhenti ketika tidak ada lagi *neighbor* yang lebih baik. Algoritma ini sederhana dan cepat tetapi mudah terjebak pada *local maximum*. Terdapat beberapa varian hill-climbing, salah satunya adalah steepest ascent yang digunakan di eksperimen ini. **Steepest Ascent** mengevaluasi seluruh neighbor saat ini, lalu berpindah ke *neighbor* dengan nilai tertinggi yang nilainya lebih tinggi dari state saat ini.

2.3 Simulated Annealing

Simulated annealing adalah algoritma optimasi yang terinspirasi dari proses annealing pada logam, yaitu pemanasan hingga suhu tinggi lalu pendinginan secara perlahan agar material mencapai struktur dengan energi rendah.

2.4 Genetic Algorithm

Genetic algorithm adalah algoritma pencarian yang meniru mekanisme seleksi alam dan genetika.

PEMBAHASAN

3.1 Struktur Program

```
IF3070_G59/
├── data/
│   └── data_uji.json          data kapal dan kendaraan
├── src/
│   ├── main.py                program utama, menjalankan seluruh eksperimen
│   ├── state.py               representasi state, initial state acak, tampilan geladak
│   ├── objective.py           objective function dan pengecekan constraint
│   ├── langkah.py             langkah neighbor: swap, pindahkan, keluarkan, rotate
│   ├── hill_climbing.py       Steepest Ascent Hill-climbing
│   ├── simulated_annealing.py Simulated Annealing
│   └── genetic_algorithm.py   Genetic Algorithm
└── output/                    hasil run (state awal/akhir dan grafik)
```

3.2 Implementasi

3.2.1 data_uji.json

```
{
  "kapal": {
    "Dimensi": {"w": 4, "l": 4},
    "Luas": 16,
    "MaxCapacity": 7
  },
  "kendaraan": [
    {"id": "motor_1",      "Jenis": "motor",       "Dimensi": {"w": 1,
"l": 2}, "Weight": 1, "ShippingFee": 8,  "Eta": 2},
    {"id": "motor_2",      "Jenis": "motor",       "Dimensi": {"w": 1,
"l": 2}, "Weight": 1, "ShippingFee": 6,  "Eta": 4},
    {"id": "mobilkecil_1", "Jenis": "mobil kecil", "Dimensi": {"w": 2,
"l": 2}, "Weight": 2, "ShippingFee": 15, "Eta": 3},
    {"id": "mobilkecil_2", "Jenis": "mobil kecil", "Dimensi": {"w": 2,
"l": 2}, "Weight": 2, "ShippingFee": 18, "Eta": 1},
    {"id": "mobilbesar_1", "Jenis": "mobil besar", "Dimensi": {"w": 2,
"l": 3}, "Weight": 3, "ShippingFee": 30, "Eta": 2},
    {"id": "mobilbesar_2", "Jenis": "mobil besar", "Dimensi": {"w": 2,
"l": 3}, "Weight": 3, "ShippingFee": 25, "Eta": 5}
  ]
}
```

3.2.2 state.py

```
import random

def ukuran(k, o):
    w,l = k["Dimensi"]["w"], k["Dimensi"]["l"]
    if o == "H":
        return l, w
    else:
        return w, l

def state_awal_acak(kendaraan, kapal):
    terisi, berat, state = set(), 0, {}
    for k in random.sample(kendaraan, len(kendaraan)):
        o = random.choice(["H", "V"])
        lebar, tinggi = ukuran(k,o)
        x = random.randint(0, kapal["Dimensi"]["l"] - lebar)
        y = random.randint(0, kapal["Dimensi"]["w"] - tinggi)
```

```
        sel = {(x + i, y + j) for i in range(lebar) for j in
range(tinggi)}

        menimpa = False
        for s in sel:
            if s in terisi:
                menimpa = True

        if menimpa or berat + k["Weight"] > kapal["MaxCapacity"]:
            state[k["id"]] = None
        else:
            state[k["id"]] = (x, y, o)
            terisi.update(sel)
            berat += k["Weight"]
    return state


def show_geladak(state, kendaraan, kapal):
    geladak = [["."] * kapal["Dimensi"]["l"] for _ in
range(kapal["Dimensi"]["w"])]
    for k in kendaraan:
        if state[k["id"]] is None:
            continue
        x, y, o = state[k["id"]]
        lebar, tinggi = ukuran(k,o)
        for j in range(tinggi):
            for i in range(lebar):
                geladak[y + j][x + i] = k["id"]
    for baris in geladak:
        print(" ".join(f"{s:>12}" for s in baris))
```

3.2.3 objective.py

```
from state import ukuran

def nilai(state, kendaraan):
    total = 0
    for k in kendaraan:
        if state[k["id"]] is not None:
            total += k["ShippingFee"]
    return total
```

```
def valid(state, kendaraan, kapal):
    terisi, berat = set(), 0
    for k in kendaraan:
        if state[k["id"]] is None:
            continue
        x, y, o = state[k["id"]]
        lebar, tinggi = ukuran(k, o)
        if x < 0 or y < 0 or x + lebar > kapal["Dimensi"]["l"] or y +
tinggi > kapal["Dimensi"]["w"]:
            return False
        sel = {(x + i, y + j) for i in range(lebar) for j in
range(tinggi)}
        for s in sel:
            if s in terisi:
                return False
        terisi.update(sel)
        berat += k["Weight"]
    return berat <= kapal["MaxCapacity"]
```

3.2.4 langkah.py

```
def swap(state, id_a, id_b):
    baru = dict(state)
    baru[id_a], baru[id_b] = baru[id_b], baru[id_a]
    return baru

def pindahkan(state, id_k, x, y, o):
    baru = dict(state)
    baru[id_k] = (x, y, o)
    return baru

def keluarkan(state, id_k):
    baru = dict(state)
    baru[id_k] = None
    return baru

def rotate(state, id_k):
    baru = dict(state)
    x, y, o = baru[id_k]
    baru[id_k] = (x, y, "V" if o == "H" else "H")
```

```
    return baru
```

3.2.5 hill_climbing.py

```
import time
from state import state_awal_acak, ukuran
from objective import nilai, valid
from langkah import swap, pindahkan, keluarkan, rotate


def semua_neighbor(state, kendaraan, kapal):
    hasil = []
    ids = list(state)

    for a in range(len(ids)):
        for b in range(a + 1, len(ids)):
            if state[ids[a]] != state[ids[b]]:
                hasil.append(swap(state, ids[a], ids[b]))

    for k in kendaraan:
        for o in ["H", "V"]:
            lebar, tinggi = ukuran(k, o)
            for y in range(kapal["Dimensi"]["w"] - tinggi + 1):
                for x in range(kapal["Dimensi"]["l"] - lebar + 1):
                    if state[k["id"]] != (x, y, o):
                        hasil.append(pindahkan(state, k["id"], x, y, o))

    for id_k in ids:
        if state[id_k] is not None:
            hasil.append(keluarkan(state, id_k))
            hasil.append(rotate(state, id_k))

    return [n for n in hasil if valid(n, kendaraan, kapal)]


def steepest_ascent(kendaraan, kapal):
    mulai = time.time()
    state = state_awal_acak(kendaraan, kapal)
    awal = state
    nilai_sekarang = nilai(state, kendaraan)
    riwayat = [nilai_sekarang]
```

```
    iterasi = 0

    while True:
        terbaik = None
        nilai_terbaik = nilai_sekarang
        for n in semua_neighbor(state, kendaraan, kapal):
            if nilai(n, kendaraan) > nilai_terbaik:
                terbaik = n
                nilai_terbaik = nilai(n, kendaraan)

        if terbaik is None:
            break

        state = terbaik
        nilai_sekarang = nilai_terbaik
        riwayat.append(nilai_sekarang)
        iterasi += 1

    return {
        "awal": awal,
        "akhir": state,
        "nilai_akhir": nilai_sekarang,
        "riwayat": riwayat,
        "iterasi": iterasi,
        "durasi": time.time() - mulai,
    }
```

3.2.6 simulated_annealing.py

Parameter yang digunakan

parameter	nilai
Suhu awal	100
Laju pendinginan	0.99
Suhu akhir	0.01

```
import math
import random
import time
```

```python
from state import state_awal_acak
from objective import nilai
from hill_climbing import semua_neighbor


def simulated_annealing(kendaraan, kapal, suhu_awal=100,
laju_pendinginan=0.99, suhu_akhir=0.01):
    mulai = time.time()
    state = state_awal_acak(kendaraan, kapal)
    awal = state
    nilai_sekarang = nilai(state, kendaraan)
    terbaik, nilai_terbaik = state, nilai_sekarang
    riwayat, probabilitas = [nilai_sekarang], []
    stuck = 0
    suhu = suhu_awal
    tetangga = semua_neighbor(state, kendaraan, kapal)

    while suhu > suhu_akhir:
        if all(nilai(n, kendaraan) <= nilai_sekarang for n in tetangga):
            stuck += 1

        n = random.choice(tetangga)
        delta = nilai(n, kendaraan) - nilai_sekarang

        if delta > 0:
            terima = True
        else:
            p = math.exp(delta / suhu)
            probabilitas.append((len(riwayat) - 1, p))
            terima = random.random() < p

        if terima:
            state = n
            nilai_sekarang = nilai(n, kendaraan)
            tetangga = semua_neighbor(state, kendaraan, kapal)
            if nilai_sekarang > nilai_terbaik:
                terbaik, nilai_terbaik = state, nilai_sekarang

        riwayat.append(nilai_sekarang)
        suhu *= laju_pendinginan
```

```
    return {
        "awal": awal,
        "akhir": terbaik,
        "nilai_akhir": nilai_terbaik,
        "riwayat": riwayat,
        "probabilitas": probabilitas,
        "stuck": stuck,
        "iterasi": len(riwayat) - 1,
        "durasi": time.time() - mulai,
    }
```

3.2.7 genetic_algorithm.py

Parameter yang digunakan

Parameter	Nilai
Populasi (kontrol)	20
Variasi populasi	10, 20, 40
Iterasi (kontrol)	30
Variasi iterasi	`10, 30, 60

```
import random
import time
from state import state_awal_acak, ukuran
from objective import nilai
from hill_climbing import semua_neighbor


def perbaiki(state, kendaraan, kapal):
    baru, terisi, berat = {}, set(), 0
    for k in kendaraan:
        posisi = state[k["id"]]
        if posisi is None:
            baru[k["id"]] = None
            continue
        x, y, o = posisi
```

```
        lebar, tinggi = ukuran(k, o)
        sel = {(x + i, y + j) for i in range(lebar) for j in
range(tinggi)}
        menimpa = False
        for s in sel:
            if s in terisi:
                menimpa = True
        if menimpa or berat + k["Weight"] > kapal["MaxCapacity"]:
            baru[k["id"]] = None
        else:
            baru[k["id"]] = posisi
            terisi.update(sel)
            berat += k["Weight"]
    return baru


def seleksi(populasi, kendaraan):
    bobot = [nilai(s, kendaraan) + 1 for s in populasi]
    return random.choices(populasi, weights=bobot, k=1)[0]


def crossover(ayah, ibu, kendaraan, kapal):
    titik = random.randint(1, len(kendaraan) - 1)
    anak = {}
    for i, k in enumerate(kendaraan):
        anak[k["id"]] = ayah[k["id"]] if i < titik else ibu[k["id"]]
    return perbaiki(anak, kendaraan, kapal)


def mutasi(state, kendaraan, kapal):
    return random.choice(semua_neighbor(state, kendaraan, kapal))


def genetic_algorithm(kendaraan, kapal, jumlah_populasi, banyak_iterasi):
    mulai = time.time()
    populasi = [state_awal_acak(kendaraan, kapal) for _ in
range(jumlah_populasi)]
    awal = max(populasi, key=lambda s: nilai(s, kendaraan))
    riwayat_maks, riwayat_rata = [], []
```

```
    for _ in range(banyak_iterasi):
        nilai_populasi = [nilai(s, kendaraan) for s in populasi]
        riwayat_maks.append(max(nilai_populasi))
        riwayat_rata.append(sum(nilai_populasi) / len(nilai_populasi))

        terbaik = max(populasi, key=lambda s: nilai(s, kendaraan))
        populasi_baru = [terbaik]
        while len(populasi_baru) < jumlah_populasi:
            ayah = seleksi(populasi, kendaraan)
            ibu = seleksi(populasi, kendaraan)
            anak = crossover(ayah, ibu, kendaraan, kapal)
            anak = mutasi(anak, kendaraan, kapal)
            populasi_baru.append(anak)
        populasi = populasi_baru

    nilai_populasi = [nilai(s, kendaraan) for s in populasi]
    riwayat_maks.append(max(nilai_populasi))
    riwayat_rata.append(sum(nilai_populasi) / len(nilai_populasi))
    akhir = max(populasi, key=lambda s: nilai(s, kendaraan))
    return {
        "awal": awal,
        "akhir": akhir,
        "nilai_akhir": nilai(akhir, kendaraan),
        "riwayat_maks": riwayat_maks,
        "riwayat_rata": riwayat_rata,
        "populasi": jumlah_populasi,
        "iterasi": banyak_iterasi,
        "durasi": time.time() - mulai,
    }
```

3.2.8 main.py

```
import json
from objective import nilai
from hill_climbing import steepest_ascent
from simulated_annealing import simulated_annealing
from genetic_algorithm import genetic_algorithm

with open("data/data_uji.json") as f:
    data = json.load(f)
kapal = data["kapal"]
```

```
kendaraan = data["kendaraan"]



def lapor(nama, hasil):
    teks = f"{nama:22} awal = {nilai(hasil['awal'], kendaraan):3}   akhir
= {hasil['nilai_akhir']:3}   iterasi = {hasil['iterasi']:4}   durasi =
{hasil['durasi']:.4f} s"
    if "stuck" in hasil:
        teks += f"   stuck = {hasil['stuck']}"
    if "populasi" in hasil:
        teks += f"   populasi = {hasil['populasi']}"
    print(teks)



for run in range(1, 4):
    lapor(f"steepest_run{run}", steepest_ascent(kendaraan, kapal))

for run in range(1, 4):
    lapor(f"sa_run{run}", simulated_annealing(kendaraan, kapal))

for populasi, iterasi in [(20, 10), (20, 30), (20, 60), (10, 30), (40,
30)]:
    for run in range(1, 4):
        lapor(f"ga_pop{populasi}_iter{iterasi}_run{run}",
genetic_algorithm(kendaraan, kapal, populasi, iterasi))
```

HASIL DAN ANALISIS

4.1 Hasil Eksekusi Program

steepest_run1 awal = 41 akhir = 63 iterasi = 1 durasi = 0.0011 s
steepest_run2 awal = 31 akhir = 63 iterasi = 2 durasi = 0.0019 s
steepest_run3 awal = 39 akhir = 44 iterasi = 1 durasi = 0.0018 s
sa_run1 awal = 41 akhir = 63 iterasi = 917 durasi = 0.3534 s stuck = 656
sa_run2 awal = 26 akhir = 62 iterasi = 917 durasi = 0.3330 s stuck = 659
sa_run3 awal = 38 akhir = 63 iterasi = 917 durasi = 0.2889 s stuck = 550
ga_pop20_iter10_run1 awal = 61 akhir = 62 iterasi = 10 durasi = 0.0879 s populasi = 20
ga_pop20_iter10_run2 awal = 55 akhir = 63 iterasi = 10 durasi = 0.0849 s populasi = 20
ga_pop20_iter10_run3 awal = 51 akhir = 59 iterasi = 10 durasi = 0.0853 s populasi = 20
ga_pop20_iter30_run1 awal = 63 akhir = 63 iterasi = 30 durasi = 0.2761 s populasi = 20

ga_pop20_iter30_run2 awal = 56 akhir = 63 iterasi = 30 durasi = 0.2584 s populasi = 20
ga_pop20_iter30_run3 awal = 51 akhir = 63 iterasi = 30 durasi = 0.2547 s populasi = 20
ga_pop20_iter60_run1 awal = 57 akhir = 63 iterasi = 60 durasi = 0.5158 s populasi = 20
ga_pop20_iter60_run2 awal = 58 akhir = 63 iterasi = 60 durasi = 0.4971 s populasi = 20
ga_pop20_iter60_run3 awal = 56 akhir = 63 iterasi = 60 durasi = 0.4583 s populasi = 20
ga_pop10_iter30_run1 awal = 48 akhir = 63 iterasi = 30 durasi = 0.1107 s populasi = 10
ga_pop10_iter30_run2 awal = 48 akhir = 63 iterasi = 30 durasi = 0.1070 s populasi = 10
ga_pop10_iter30_run3 awal = 56 akhir = 63 iterasi = 30 durasi = 0.1151 s populasi = 10
ga_pop40_iter30_run1 awal = 53 akhir = 63 iterasi = 30 durasi = 0.5051 s populasi = 40
ga_pop40_iter30_run2 awal = 62 akhir = 63 iterasi = 30 durasi = 0.5221 s populasi = 40
ga_pop40_iter30_run3 awal = 56 akhir = 63 iterasi = 30 durasi = 0.5256 s populasi = 40

4.2 Hasil Hill Climbing

Run	Nilai awal	Nilai akhir	Banyak iterasi	Durasi (detik)
1	41	63	1	0.0011
2	31	63	2	0.0019
3	39	44	1	0.0018

4.3 Hasil Simulated Annealing

Run	Nilai awal	Nilai akhir	Banyak iterasi	Frekuensi Stak	Durasi (detik)
1	41	63	917	656	0.3543
2	26	62	917	659	0.3330
3	38	63	917	550	0.2889

4.4 Hasil Genetic Algorithm

ga_pop20_iter10_run1 awal = 61 akhir = 62 iterasi = 10 durasi = 0.0879 s populasi = 20
ga_pop20_iter10_run2 awal = 55 akhir = 63 iterasi = 10 durasi = 0.0849 s populasi = 20
ga_pop20_iter10_run3 awal = 51 akhir = 59 iterasi = 10 durasi = 0.0853 s populasi = 20
ga_pop20_iter30_run1 awal = 63 akhir = 63 iterasi = 30 durasi = 0.2761 s populasi = 20
ga_pop20_iter30_run2 awal = 56 akhir = 63 iterasi = 30 durasi = 0.2584 s populasi = 20
ga_pop20_iter30_run3 awal = 51 akhir = 63 iterasi = 30 durasi = 0.2547 s populasi = 20
ga_pop20_iter60_run1 awal = 57 akhir = 63 iterasi = 60 durasi = 0.5158 s populasi = 20
ga_pop20_iter60_run2 awal = 58 akhir = 63 iterasi = 60 durasi = 0.4971 s populasi = 20
ga_pop20_iter60_run3 awal = 56 akhir = 63 iterasi = 60 durasi = 0.4583 s populasi = 20
ga_pop10_iter30_run1 awal = 48 akhir = 63 iterasi = 30 durasi = 0.1107 s populasi = 10

ga_pop10_iter30_run2 awal = 48 akhir = 63 iterasi = 30 durasi = 0.1070 s populasi = 10
ga_pop10_iter30_run3 awal = 56 akhir = 63 iterasi = 30 durasi = 0.1151 s populasi = 10
ga_pop40_iter30_run1 awal = 53 akhir = 63 iterasi = 30 durasi = 0.5051 s populasi = 40
ga_pop40_iter30_run2 awal = 62 akhir = 63 iterasi = 30 durasi = 0.5221 s populasi = 40
ga_pop40_iter30_run3 awal = 56 akhir = 63 iterasi = 30 durasi = 0.5256 s populasi = 40

4.5 Hasil Akhir

Algoritma	Nilai akhir rata-rata	Nilai akhir minimum	Nilai akhir maksimum	Durasi rata-rata (detik)
Hill Climbing Steepest Ascent	56.67	44	63	0.0016
Simulated Annealing	62.67	62	63	0.3251
Genetic Algorithm populasi 20, iterasi 10	61.33	59	63	0.0860
Genetic Algorithm populasi 20, iterasi 30	63	63	63	0.2631
Genetic Algorithm populasi 20, iterasi 60	63	63	63	0.4904
Genetic Algorithm populasi 10, iterasi 30	63	63	63	0.1109
Genetic Algorithm populasi 40, iterasi 30	63	63	63	0.5176

4.6 Analisis

Hill Climbing paling cepat (0.0016 s) tetapi tidak stabil (44-63).

Simulated Annealing lebih stabil (62=63) tetapi lebih lambat (0.3251 s).

Genetic Algorithm dengan iterasi lebih atau sama dengan 30 paling baik, karena selalu menghasilkan 63.

KESIMPULAN DAN SARAN

5.1 Kesimpulan

1. Hill Climbing merupakan algoritma tercepat

2. Genetic Algorithm dengan iterasi lebih atau sama dengan 30 menghasilkan hasil paling baik

5.2 Saran

1. Menguji dengan data yang lebih besar dan banyak. Pada eksperimen kali ini kami menggunakan kapal dengan grid hanya 4x4 dan jumlah total kendaraan hanya 6 buah. Data yang lebih besar dan banyak akan menghasilkan perbandingan yang lebih jelas antar algoritma local search sehingga menghasilkan kesimpulan yang lebih kuat dan akurat.

2. Melakukan eksekusi program yang lebih banyak. Eksperimen kali ini melakukan 3 run untuk setiap algoritma, terkadang hasilnya belum mencapai optimum. Oleh karena itu, semakin banyak eksekusi program maka hasilnya akan semakin mendekati sempurna.

3. Mengimplementasi varian Hill Climbing yang lain, yaitu Sideways, Stochastic, dan Random Restart. Untuk bisa membandingkan hasil dari berbagai jenis algoritma.

PUSTAKA

S. Russell dan P. Norvig, Artificial Intelligence: A Modern Approach
, ed. ke-4, GlobalEdition. Harlow: Pearson, 2021, Bab 4 “Search in Complex Environments”.

LAMPIRAN

LAMPIRAN A. Form Penggunaan Generative AI

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, saya/kami*:

Nama/NIM	:	M. Dhafin Fadhlillah / 18223103
Fakultas/Sekolah	:	18223103
Kode Mata Kuliah	:	IF 3070
Nama Mata Kuliah	:	Dasar Interaksi Artifisial
Judul Tugas/Proposal	:	Tugas Besar 1

Menyatakan bahwa saya/kami* menggunakan/tidak menggunakan* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	~~Ya~~/Tidak*	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	~~Ya~~/Tidak*	
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	~~Ya~~/Tidak*	
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	~~Ya~~/Tidak*	
Lainnya, Jelaskan:		

Saya/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

(tempat), (tanggal)
(tanda tangan)
(Nama Lengkap)
NIM

Dhafin
18223103

19

LAMPIRAN B. Tautan Repository

Repository: https://github.com/dhafin0/IF3070_G59

LAMPIRAN C. Pembagian Tugas

NIM	NAMA	Kontribusi
18223103	Muhammad Dhafin Faidhulhaq	Tugas Besar 1

LAMPIRAN D. Bukti Run Program

